44. Reliability: Timeouts, Circuit Breakers, Graceful Shutdown

44. Reliability: Timeouts, Circuit Breakers, Graceful Shutdown

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `44-reliability`

Learning objectives

- Propagate deadlines everywhere
- Implement circuit breaker behavior
- Shut down HTTP servers gracefully
- Define SLOs mentally

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Reliability patterns — timeouts, circuit breakers, graceful shutdown — keep incidents small and deploys safe.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Timeouts everywhere: clients, servers, DB.
- Circuit breakers stop calling unhealthy deps.
- Graceful shutdown: stop intake, drain, then exit.
- signal.Notify for SIGTERM in containers.
- Load shedding beats cascading failure.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-reliability: && cd /tmp/gw-reliability:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    stop := make(chan struct{})
    go func() {
        time.Sleep(20 * time.Millisecond)
        close(stop)
    }()
    select {
    case <-stop:
        fmt.Println("shutdown")
    case <-time.After(time.Second):
        fmt.Println("timeout")
    }
}
```

Run it:

```
go run .
```

Expected output:

```
shutdown
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Simulate SIGTERM with a cancel channel.
2. Add a timeout around a slow function.
3. Sketch circuit breaker states: closed/open/half-open.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Timeouts

Every external call needs a deadline. Inherit from request context when serving HTTP.

Circuit breaker

When dependency error rate spikes, fail fast temporarily to allow recovery — then probe half-open.

Graceful shutdown

```
go func() { _ = srv.ListenAndServe() }()
<-stop
ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
_ = srv.Shutdown(ctx)
```

Drain in-flight requests; stop accepting new ones.

SLO mindset

Availability and latency targets drive timeout budgets and retry policies.

Failure modes to watch while coding

- Kill –9 as the only deploy strategy
- No timeout on outbound calls
- Circuit breaker without metrics

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- signal handling similar; asyncio cancellation related.
- Flask often lacks std graceful shutdown — learn Go's.
- Fail-fast vs retry — product decision.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Kubernetes sends SIGTERM then SIGKILL — drain within terminationGracePeriodSeconds.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Reliability: Timeouts, Circuit Breakers, Graceful Shutdown" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Kill –9 as the only deploy strategy
- No timeout on outbound calls
- Circuit breaker without metrics

Exercises

Work in `exercises/44-reliability/`. Read that folder's `README.md` before you code.

What you will build

Implement shutdown/timeout helpers so tests observe clean stops.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/44-reliability/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does Shutdown do?
2. Fail-fast purpose?
3. Timeout budget meaning?

Further reading

- https://pkg.go.dev/net/http#Server.Shutdown